به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴) ۱۲ اردیبهشت ۹۸

دانشگاه صنعتی شریف مدرس: مهدی صفرنژاد

اعداد بزرگ و دیگر داده‌ساختارها

سؤال ۱. [۴۵ نمره] دیگر داده‌ساختارها

آ. [۱۰ نمره] الگوریتمی با استفاده از پشته[۱] ارائه دهید که یک آرایه به طول n مثل $A[۰, n-۱]$ و یک عدد k را از ورودی دریافت کند و کمینهٔ هر k عنصر متوالی را در زمان خطی پیدا کند. به طور دقیق‌تر خروجی الگوریتم یک آرایه به طول $n-k+۱$ مثل $B[۰, n-k]$ است که $B[i] = \min\{A[i], A[i+۱], \ldots, A[i+k-۱]\}$. (راهنمایی: عناصر B را یکی یکی حساب کنید. برای استفاده از اطلاعات مراحل قبل در محاسبهٔ عنصر $B[i]$ فعلی، باید علاوه بر $B[i-۱]$، اطلاعات دیگری را نیز نگه‌داری کنید. چه اطلاعاتی بهتر است نگه‌داری شود و چرا پشته برای نگه‌داری این اطلاعات پشته کفایت می‌کند؟)

پاسخ: روی عناصر آرایه حرکت می‌کنیم و وقتی به عنصر $A[i]$ می‌رسیم، آن را در یک پشته push می‌کنیم با این تفاوت که قبل از push کردن تا جای ممکن اگر عنصر بالای پشته بیشتر از $A[i]$ بود آن را pop می‌کنیم. توجه کنید با این کار از پایین به بالای پشته عناصر مقدارشان صعودی است و اندیس مربوط به آن‌ها نیز صعودی است و بالاترین عنصر در هر لحظه $A[i]$ است. همچنین در هر لحظه اگر $i \geq k-۱$ بود، می‌خواهیم کمترین عنصر $A[i-k+۱ \ldots i]$ را خروجی دهیم. برای این کار به پایین‌ترین عنصر پشته نگاه می‌کنیم و اگر خارج از این بازه بود آن را حذف می‌کنیم و در نهایت پایین‌ترین عنصر پشته را به عنوان عنصر کمینه خروجی می‌دهیم. (با توجه به این حذف، می‌توان گفت داده‌ساختار استفاده شده صف دوطرفه است و نه پشته. توجه کنید تأکید بر روی الگوریتم است و توجه به این نکته تأثیری در نمره ندارد.)

در این صورت در هر مرحله یک push، تعدادی pop و حداکثر یک حذف از پایین پشته داریم. توجه کنید تعداد در مجموع popها حداکثر n است و بنابراین زمان الگوریتم در مجموع $O(n)$ خواهد بود.

ب. [۵ نمره] درخت متناظر عبارت پسوندی زیر را ترسیم کند. سپس عبارت پیشوندی متناظر آن را بنویسید.

$$a\ b + c\ d - * e\ f + /$$

پاسخ: $/ * + a\ b - c\ d + e\ f$

پ. [۱۰ نمره] تریپ (treap) که ترکیب درخت (tree) و هرم (heap) است، یک درخت دودویی جست‌وجو تصادفی است که ارتفاع آن به احتمال زیاد $\Theta(\log n)$ است. این داده‌ساختار را از کتاب CLRS (ویرایش سوم کتاب، تمرین ۱۳-۴، صفحهٔ ۳۳۳) مطالعه کنید و فقط به سؤال قسمت a آن پاسخ دهید.

پاسخ: هدف از این سؤال مطالعه مبحث تریپ بوده است و پاسخ قسمت a سؤال نسبتاً ساده است. ریشه r به صورت یکتا تعیین می‌شود زیرا کمترین اولویت را دارد. سپس مجموعه به وسیله کلید ریشه به دو زیرمجموعه یکی

[۱] stack

با کلیدهای کوچکتر از کلید r و یکی با کلیدهای بزرگ‌تر از کلید r افزار می‌شوند. هر کدام از این زیرمجموعه‌ها نیز یک تریپ تشکیل می‌دهند که زیردرخت‌های چپ و راست r را تشکیل می‌دهند. بنابراین با استفاده از استقرا روی تعداد رأس‌ها می‌توان ثابت کرد که ساختار تریپ به صورت یکتا تعیین می‌شود.

ت. [۱۲ نمره] از کتاب CLRS (ویرایش سوم) تمرین‌های ۱۷,۱-۳ (ص ۴۵۶)، ۱۷,۲-۲ (ص ۴۵۹) و ۱۷,۳-۲ (ص ۴۶۲) را حل کنید. سؤال در مورد تحلیل سرشکن یک داده‌ساختار فرضی با سه روش مختلف هست.

پاسخ: روش تجمیعی: مجموع هزینه‌های عملیات برابر است با

$$\sum_{i=1}^{n} c_i \le \sum_{i=1}^{n} 1 + \sum_{j=0}^{\lg n} 2^j < 3n = O(n).$$

بنابر این هزینهٔ سرشکن هر عمل برابر است با $O(n)/n = O(1)$.

روش حسابداری: ادعا می‌کنیم هر عمل هزینهٔ سرشکن ۳ دارد. اگر i توانی از ۲ نبود، یک واحد را هزینه می‌کنیم و ۲ واحد را در حساب بانکی ذخیره می‌کنیم. اگر i توانی از ۲ بود، i واحد از حساب بانکی برداشت می‌کنیم. مشابه روش تجمیعی می‌توان نشان داد که حساب بانکی منفی نمی‌شود و بنابراین هزینه سرشکن هر عمل $O(1)$ است.

روش تابع پتانسیل: تابع پتانسل را به صورت $\Phi(D_0) = 0$ و $\Phi(D_i) = 2i - 2^{1+\lfloor \lg i \rfloor}$ برای $i > 0$ تعریف می‌کنیم. برای اولین عمل داریم

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + 2i - 2^{1+\lfloor \lg i \rfloor} - 0 = 1.$$

برای اعمال بعدی اگر i توانی از ۲ نباشد داریم

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + 2i - 2^{1+\lfloor \lg 1 \rfloor} - (2(i-1) - 2^{1+\lfloor \lg(i-1) \rfloor}) = 3.$$

و اگر i توانی از ۲ باشد داریم

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = i + 2i - 2^{1+j} - (2(i-1) - 2^{1+j-1}) = i + 2i - 2i - 2i + 2 + i = 2.$$

بنابراین هزینه سرشکن هر عمل حداکثر ۳ است.

ث. [۸ نمره] با استفاده از آرایهٔ پسوندی و آرایهٔ کمکی طولانی‌ترین پیشوند مشترک (LCP) برای یک رشتهٔ s، چطور می‌توان کوچکترین زیررشته ناتهی s را پیدا کرد که فقط یک بار در s تکرار شده است؟

پاسخ: تمام خانه‌های آرایهٔ پسوندی را به صورت زیر بررسی می‌کنیم. اگر SA آرایه پسوندی باشد و $SA[i] = j$ باشد، در این صورت کوچکترین رشتهٔ یکتای s که از $s[j]$ شروع می‌شود را می‌توان با کمک LCP و مقایسه با $s[SA[i-1]]$ و $s[SA[i+1]]$ در $O(1)$ پیدا کرد. در نهایت کمترین مقدار پیدا شده را خروجی می‌دهیم.

سؤال ۲. **[۱۵ نمره]** RSA [۲] یک سیستم رمزنگاری کلید عمومی[۳] است. این روش یکی از بزرگ‌ترین پیشرفت‌ها در زمینهٔ رمزنگاری و یکی از اساس‌های امنیت اینترنت به شمار می‌رود. RSA روشی است که امنیت نشست‌های[۴] HTTPS[۵] سایت‌های تجارت الکترونیکی و خدمات ابری را فراهم می‌کند. این روش در پروتکل‌های SSL[۶] و SSH[۷] نیز استفاده می‌شود.

در این سیستم به طور کلی هر کاربر دو عدد اول تصادفی بزرگ مانند p و q تولید می‌کند. این کاربر پیمانهٔ عمومی[۸] خود را برابر $m = p \cdot q$ می‌گیرد. پس از آن کاربر یک عدد کوچک به نام e در نظر می‌گیرد که نسبت به $(p-۱)(q-۱)$ اول باشد و d را برابر با وارون ضربی e نسبت به این پیمانه می‌گیرد. به عبارت دیگر $d \cdot e = ۱ \mod (p-۱)(q-۱)$. این کاربر (e, m) را به عنوان کلید عمومی خود اعلام می‌کند و d را (به عنوان بخشی از کلید خصوصی) پیش خود نگه می‌دارد. اگر کسی بخواهد به این کاربر پیغامی مثل عدد n ارسال کند، آن را به صورت $c = E(n) = n^e \mod m$ رمزگذاری می‌کند. کاربر نیز این پیغام را به صورت $D(c) = c^d \mod m$ رمز گشایی می‌کند. بنابر قضیهٔ اویلر تقریبا برای تمام nها داریم $D(E(n)) = n$ که نشان می‌دهد پیام به روش گفته شده رمز گشایی می‌شود. با توجه به این‌که دیگر افراد عدد d را ندارند، قادر به رمزگشایی پیام نخواهند بود (مگر این که به طریقی بتوانند m را به عوامل اولش تجزیه کنند که راه سریعی برای این کار تاکنون پیدا نشده است).

آ. **[۷ نمره]** فرض کنید m یک عدد k بیتی باشد و برای محاسبهٔ $E(n)$ از الگوریتم MODULAR-EXPONENTIATION که صفحهٔ ۹۵۷ کتاب CLRS (ویرایش سوم) آمده استفاده می‌کنیم. همچنین برای ضرب دو عدد از الگوریتم کاراتسوبا که در کلاس گفته شد استفاده می‌کنیم. در این صورت مرتبهٔ زمانی رمزنگاری (محاسبهٔ $E(n)$) را محاسبه کنید.

پاسخ: فرض کنید $d = \log_۲ m$ باشد. الگوریتم MODULAR-EXPONENTIATION از $O(d)$ ضرب استفاده می‌کند. اگر هر ضرب را با الگوریتم کاراتسوبا در $O(d^{\log_۲ ۳})$ انجام دهیم، زمان اجرای رمزنگاری $O(d^{۱+\log_۲ ۳})$ است.

ب. **[۸ نمره]** فرض کنید برای پیدا کردن اعداد اول (p یا q) از الگوریتم تصادفی تست فرما (که در این لینک توضیح داده شده) برای بررسی اول بودن اعداد استفاده می‌کنیم. اگر از برخی پیچیدگی‌های الگوریتم صرف‌نظر کنیم، الگوریتم یک عدد تصادفی k رقمی مثل n انتخاب می‌شود و می‌خواهیم به صورت تصادفی بررسی کنیم که آیا این عدد اول است یا خیر. برای این بررسی مثلاً در ۱۰۰ مرحله یک عدد تصادفی a انتخاب می‌شود و $a^{n-۱} \mod n$ محاسبه می‌شود. اگر این عدد، برابر با یک نبود n حتماً مرکب است. همچنین اگر n یک عدد اول نباشد (با صرف‌نظر از اعداد کارمایکل)، برای نیمی از aهای ممکن این عدد نامساوی با یک است.

- این الگوریتم از نوع مونت‌کارلو است یا لاس‌وگاس؟ احتمال خطای آن را بیان کنید.
- با استفاده از قضیهٔ عدد اول (که در این لینک توضیح داده شده) احتمال این که عدد n انتخاب شده اول باشد (به صورت مجانبی) چقدر است؟
- زمان اجرای میانگین این روش را برای پیدا کردن یک عدد اول k رقمی بیان کنید.

---

[۲] Rivest-Shamir-Adelman
[۳] public-key
[۴] session
[۵] Hypertext Transfer Protocol Secure
[۶] Secure Sockets Layer
[۷] Secure Shell
[۸] public modulus

پاسخ: الگوریتم بررسی اول بودن یک عدد مونت‌کارلو است زیرا سریع است ولی احتمالاً درست است. توجه کنیم که این الگوریتم امکان ندارد عددی اول را مرکب اعلام کند، ولی این احتمال وجود دارد که عددی مرکب را اول اعلام کند (البته توجه کنیم که این الگوریتم به هیچ صورت نمی‌تواند اعداد کارمایکل را از اعداد اول تشخیص دهد و آن‌ها را در راه‌حل این سؤال در نظر نمی‌گیریم). خطای این الگوریتم آن است که ممکن است عددی مرکب را اول در نظر بگیرد. بنا به صورت سؤال می‌دانیم برای حداقل نیمی از a های ممکن $a^{n-1} \neq 1 \mod n$. پس (از آن‌جا که ۱۰۰ مرحله از یکدیگر مستقل هستند داریم)

$$\text{احتمال خطا در یک مرحله} = \text{احتمال خطا}^{100} \leq (\frac{1}{2})^{100} = 2^{-100}$$

بنا بر قضیهٔ عدد اول، اگر $pi(n)$ تعداد اعداد اول کوچکتر مساوی n باشد داریم $\pi(n) \sim \frac{n}{\ln n}$. بنابراین احتمال اول بودن n از مرتبه $\pi(n)/n = 1/\ln n$ است.

برای پیدا کردن میانگین زمان اجرای الگوریتم دقت کنید احتمال آن که این الگوریتم یک عدد را انتخاب و آن را اول اعلام کند با احتساب خطا نیز $1/\ln n$ است. در نتیجه امید ریاضی مراحل لازم برای یافتن عدد اول برابر است با $\frac{1}{O(\frac{1}{\ln n})} = O(\ln n) = O(d)$. همچنین توجه کنیم برای هر تست ۱۰۰ بار عملیات به توان رساندن را انجام می‌دهیم که با توجه به قسمت قبل به توان رساندن در زمان $\mathcal{O}(k^{1+\log_2 3})$ انجام می‌شود و ۱۰۰ هم عددی ثابت است. پس کل زمان لازم به صورت مجانبی برابر است با $\mathcal{O}(k^{2+\log_2 3})$.

سؤال ۳. [۲۵ نمره] سیل‌های نوروزی بسیاری از راه‌های ارتباطی را مسدود کرد. در این مسئله n روستا داریم که راه‌های ارتباطی آن فاقد دور است. در هر مرحله با بالا آمدن سیل، یکی از راه‌های ارتباطی قطع می‌شود. در هر مرحله شما باید اندازهٔ بزرگترین زیرمجموعه از روستاها را محاسبه کنید که به هم هنوز مسیر دارند.

در خط اول ورودی، تعداد روستاها n ($۱ \le n \le ۱,۰۰۰,۰۰۰$) و تعداد راه‌های ارتباطی آن‌ها m داده شده است. سپس در m خط بعدی، در هر خط دو عدد u و v آمده است که u و v به هم راه دوطرفه دارند. روستاها از ۱ تا n شماره‌گذاری شده‌اند. سپس در خط بعدی عدد q آمده و در q خط بعدی شمارهٔ راه‌ارتباطی‌ای که قطع می‌شود آمده است. راه‌ها به ترتیبی که در ورودی آمده‌اند شماره‌گذاری می‌شوند. خروجی باید q خط داشته باشد که در هر خط اندازهٔ بزرگترین زیرمجموعه از روستاها که پس از قطع شدن راه ارتباطی آن مرحله هنوز به هم مسیر دارند آمده باشد.

راهنمایی: توجه کنید که کل ورودی از ابتدا به شما داده شده است و نیاز نیست سؤال‌ها را به ترتیب جواب دهید. در این سؤال شما باید داده‌ساختار مجموعه‌های مجزا که در کلاس مطرح شد را پیاده‌سازی و از آن استفاده کنید.

ورودی	خروجی
5 4 1 2 2 3 1 4 1 5 2 1 4	3 2

پاسخ: با توجه به اینکه کلیه اطلاعات مسئله از ابتدا در اختیار برنامه قرار دارد، می‌توان اتفاقات را از انتها به ابتدا پاسخ داد. در این صورت در ابتدا یک گراف احتمالاً ناهمبند داریم که در هر مرحله یکی این که اندازهٔ بزرگترین مؤلفهٔ همبندی را باید خروجی دهید و دوم این‌که یک یال به گراف اضافه می‌شود. اضافه کردن یال با داده ساختار مجموعه‌های مجزا به راحتی قابل انجام است و به روز رسانی اندازهٔ بزرگترین مؤلفهٔ همبندی با نگه‌داری size (مثل rank) برای نماینده هر مجموعه و به روز رسانی آن در زمان ادغام دو مجموعه قابل انجام است.

کد پایتون این سؤال ضمیمه شده است.

سؤال ۴. [۲۵ نمره] در این سؤال روی محور xها تعدادی بازه قرار دارد که هر بازه یک وزن دارد. در هر مرحله یا یک بازه اضافه می‌شود و یا یک نقطه روی محور داده می‌شود و برنامهٔ شما باید مجموع وزن بازه‌هایی که این نقطه آن‌ها را قطع می‌کند را برگرداند. توجه کنید اگر نقطه سر یا ته بازه باشد آن را قطع نمی‌کند.

در هر خط ورودی یا `add a b w` داده می‌شود که به معنی این است که بازهٔ $[a, b]$ با وزن w اضافه شود. همچنین `ask p` نقطهٔ p را مورد پرسش قرار می‌دهد. a، b، w و p همه اعداد صحیح مثبت و کمتر از ۱,۰۰۰,۰۰۰ هستند.

شما در این سؤال باید از داده‌ساختار درخت فنویک (درخت اندیس داده شدهٔ دودویی) که در کلاس مطرح شد استفاده کنید.

ورودی	خروجی
`add 1 10 50`	150
`add 5 20 100`	100
`ask 6`	60
`ask 10`	160
`add 2 8 10`	
`ask 3`	
`ask 6`	

پاسخ: برای راحتی اندیس‌ها را در ۲ ضرب می‌کنیم و هر عمل را با استفاده از درخت فنویک به صورت زیر انجام می‌دهیم. عمل `add a b w` به دو عمل `update(2a+1, w)` و `update(2b-1, -w)` در درخت فنویک تبدیل می‌شود. عمل `ask p` نیز به عمل `sum(2p)` در درخت فنویک تبدیل می‌شود.

کد پایتون این سؤال ضمیمه شده است.

سؤال ۵. [۲۰ نمره] بلاک‌چین. در این سوال شما باید سیستم ساده شده‌ای از بلاک‌چین یا زنجیرهٔ بلوکی را پیاده‌سازی کنید. ابتدا برای آشنایی با بلاک‌چین پیشنهاد می‌شود از این لینک استفاده کنید. سرآیند هر بلوک ۸۰ کاراکتر اول آن را تشکیل می‌دهد و شامل شش فیلد زیر می‌باشد.

نام فیلد	کاربرد	طول (بایت)
Version	نسخهٔ بلوک	۴
hashPrevBlock	درهم‌سازی ۲۵۶ بیتی بلوک قبلی	۳۲
hashMerkleRoot	درهم‌سازی ۲۵۶ بیتی داده‌های بلوک	۳۲
Time	زمان حال به ثانیه	۴
Bits	میزان سختی تولید بلاک	۴
Nonce	یک عدد دلخواه	۴

بلوک‌ها یک لیست پیوندی را تشکیل می‌دهند که بلاک‌چین یا زنجیرهٔ بلوکی نام دارد. هر بلوک این زنجیره با فیلد hashPrevBlock به بلوک قبلی خود متصل می‌شود. در این سؤال روی یک بلوک از زنجیره تمرکز می‌کنیم. بعد از جمع‌آوری اطلاعات اصلی یک بلوک، سرآیند آن تشکیل می‌شود که ۵ فیلد اول آن مشخص است. برای این که بلاک مجوز اضافه شدن به زنجیره را داشته باشد، باید فیلد ششم یا nonce به صورتی انتخاب شود که اگر تابع SHA-256 را دوبار روی سرآیند بلوک اجرا کنیم، به تعداد خاصی صفر اول آن ظاهر شود. این شرط به این منظور گذاشته شده که اضافه کردن بلوک جدید به زنجیره کار سختی باشد و با وجود تلاش جهانی، حدوداً هر ۱۰ دقیقه یک بلوک به زنجیره اضافه شود. در ورودی به شما ۷۶ کاراکتر اول سرآیند داده می‌شود و وظیفهٔ شما پیدا کردن nonce مناسب و تکمیل بلوک است. به این کار ماینینگ گفته می‌شود. سرآیند در ورودی به صورت هگز (مبنای ۱۶) داده شده است و بنابراین به صورت ۱۵۲ کاراکتر ظاهر می‌شود. در خروجی نیز nonce را باید به صورت هگز بنویسید که ۴ کاراکتر آن به صورت ۸ کاراکتر هگز ظاهر می‌شود. بعد از اضافه کردن nonce، درهم‌سازی سرآیند بلوک باید ۴ تا صفر در نمایش هگز خود داشته باشد (این عدد در واقعیت ۱۸ صفر می‌باشد و به تدریج ممکن است زیاد شود).

توجه کنید ورودی و خروجی به صورت هگز هستند ولی محاسبات باید روی اطلاعات اصلی (باینری) انجام شود. همچنین در تبدیل درهم‌سازی به هگز، ممکن است رشته به صورت برعکس تبدیل شود و صفرها در برخی توابع ابتدا و یا انتهای رشته قرار گیرند. برای همین ابتدا برنامه خود را روی ورودی نمونه تست کنید. شما می‌توانید از کتابخانهٔ hashlib پایتون استفاده کنید. در این لینک بلوک کامل ورودی را می‌توانید پیدا کنید (nonce اصلی این بلوک با خروجی نمونه متفاوت است).

ورودی	خروجی
0200000066191da95594aeda1a98a19ff054a88a 510754e2a4d93e0a0000000000000008485ae79 7312b2cb37dfb1aac11d7c5ad9dd84364bbe26ff a781853996587d9b10a06555f5861618	7735c84e

درهم‌سازی ورودی+خروجی نمونه: 0000a22ba24c4bae0cfc072d79b29213364a1b339c1a78fd927d0d3a511231a0

پاسخ: در این سؤال کافی است در یک حلقه nonceهای مختلف را بررسی کنید تا جواب پیدا شود. در عین سادگی راه‌حل، این سؤال، الگوریتم استفاده شده در minerهای بلاک‌چین را نشان می‌دهد. در واقع اجرای همین الگوریتم در سال بیش از ۶۶

تراوات‌ساعت برق مصرف می‌کند که بیشتر از مصرف برق بسیاری از کشورها مثل پرتغال است. لازم به ذکر است minerها برای اجرای بهتر و سریع‌تر این الگوریتم که منجر به کشف بیت‌کوین و کسب ثروت می‌شود، سخت‌افزارهای مخصوصی طراحی و تولید کرده‌اند.

فایل `blockchain_sol.py` شامل کد راه‌حل به عنوان نمونه ضمیمه شده است.

موفق باشید